HTMLとJS

```html
<!doctype html>
<html lang="ja">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width" />

    <!--外部ファイル読み込み（css)-->
    <link rel="stylesheet" href="css/style.css">

    <title>現場猫だいすきページ</title>
  </head>
  <body>
    <h2>今日もご安全に！</h2>
    <!--<img src="images.webp" alt="テスト画像" />-->
    <p>こんにちは</p>

    <script>

      //インライン（HTMLに直接書く）

      //Windowを開いた瞬間に出てくるメッセージ
      window.alert("はじめてのJavaScript");

      //F12キーを押したときに出てくる「開発者ツール」のコンソールにメッセージ表示
      console.log("はじめてのJavaScript");

    </script>

  </body>
</html>
```

CSSは外部ファイルの読み込みで書いてある！（インラインではない）

scriptタグのなかにJSのコードを書く（インライン）

コードが長くなった場合、インラインだと見づらくなる

異なる言語が１個のファイルに同居しているのはよくない

インラインの利点	インラインの欠点
- 簡単に書ける（PATHを意識しないでいいなど） - HTMLと同じファイルなのでHTMLを参照するのが楽	- そもそも、「関心事」が絞り込めていない - 異なる言語が混在するとメンテナンスが面倒 - 片方の破損が、もう片方を巻き込むかも。。。

関心事の分離原則（SoC）

今回のコードでいえば、言語が３つあるので、それぞれの「関心事」は比較的説明や理解が簡単です。
今回は、「サイトの表示の骨格（HTML）」「サイトの骨格の肉付け（CSS）」「サイトの動きの調整（JS）」という３つの関心事に分けることができるといえそうです。
このように「何の機能があるの？」と聞かれたときなどに、答えることができる機能の粒度で考えると、関心事を抽出して分離することができます。
関心事を分離していないと、依存関係が深くなる（お互いがお互いを支えあいすぎる、これを密結合という）状態になってしまい、あとあと整理や分離がむつかしくなるケースもあります。

単一責任の原則（SRP）

すべてのコードは**「ひとつのことをうまくやる」**べきです。
なんでもできる万能なコードは、中身が複雑になりがちで、蜜結合になりすぎたりしてメンテナンスや改修が非常に困難になりがちです。なので、ひとつのファイル、ひとつのコードは、ひとつのことをうまくやることに専念するべき、という考えがあります。
この**「ひとつのことをうまくやる（＝一つの作業だけに責任を持つ）」**という考え方のことを「単一責任原則（SRP）」といいます。

```html
<!doctype html>
<html lang="ja">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width" />

    <!--外部ファイル読み込み（css)-->
    <link rel="stylesheet" href="css/style.css">

    <!--JSも外部ファイルの読み込みに修正した(defer属性済み)--
    <script src="./js/main.js" defer></script>

    <title>現場猫だいすきページ</title>
  </head>
  <body>
    <h2>今日もご安全に！</h2>
    <!--<img src="images.webp" alt="テスト画像" />-->
    <p>こんにちは</p>

    <!--
    <script>

      //インライン（HTMLに直接書く）

      //Windowを開いた瞬間に出てくるメッセージ
      window.alert("はじめてのJavaScript");

      //F12キーを押したときに出てくる「開発者ツール」のコンソールにメッセージ表示
      console.log("はじめてのJavaScript");

    </script>
    -->

  </body>
</html>
```

scriptタグで外部ファイル読み込みさせる

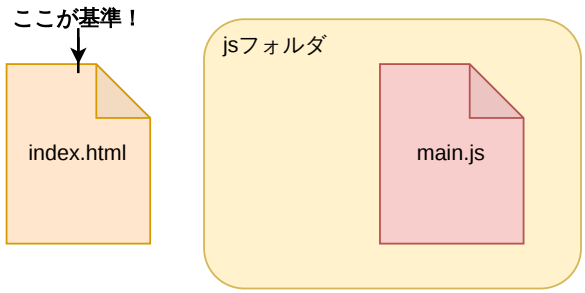
ここは消す（いったんはコメントアウトでもOK）

外部JSファイル読み込みのタグの書き方

パフォーマンス向上のおまじない

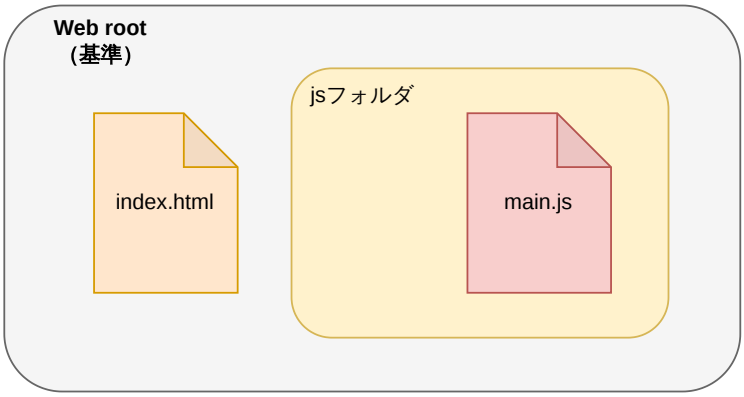
```html
<script src="./js/main.js" defer></script>
```

「このファイルから見て」（相対PATHを意味）

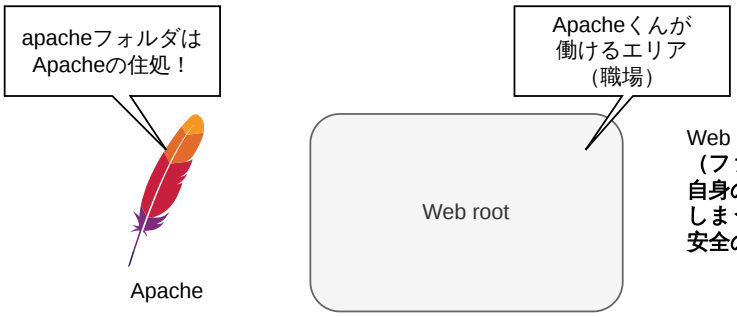


原則的に絶対PATH
を意味する
（先頭に何もつけない）

```
<script src="js/main.js" defer></script>
```



絶対PATHの場合、基本的にはWeb rootからのファイルの位置を意味します。
なぜなら、Apacheからみた全体のルートディレクトリを基準にしているからです。



Web root（職場）と住処が分かれているのは、**変化の多い職場（ファイルを自由に消したり増やしたりする）**と、**Apacheくん自身の安定した環境（ファイル欠損でApacheくん自身がしんでしまうかも？）を分けたほうが安全**だからです。
安全のために隔離するという発想じたいが、かなり重要です！